Ejercicio 1

a)
Caso A. Basta con una regla programada. Se puede comprobar si el DNI tiene ocho números y una letra mediante una expresión regular o validaciones. El aprendizaje automático no aporta una ventaja funcional porque las condiciones son concretas y fáciles de mantener.

Caso B. Está justificado utilizar aprendizaje automático. Reconocer «¡para!» a partir de audio en un taller ruidoso implica variaciones de pronunciación, acentos y ruido ambiental. Una regla manual sería insuficiente y frágil para interpretar todas esas señales.

Caso C. Basta con una regla programada. Si la entrada es texto y solo hay que comprobar si contiene «salir», se puede normalizar la cadena y buscar esa palabra. No hace falta un modelo para resolver una condición sencilla.

Caso D. Está justificado utilizar aprendizaje automático. La cámara puede captar diferentes posiciones, velocidades, ángulos e iluminación. Un modelo puede aprender patrones visuales o de coordenadas para distinguir los ejercicios; describir todas las variaciones con reglas manuales sería frágil y difícil de mantener.

b)
Aunque ambas órdenes tengan una confianza de 0,62, no deben tratarse igual porque el impacto de las acciones es distinto. Para «transfiere 500 euros a Laura», la interfaz debe pedir una confirmación antes de ejecutar la transferencia, ya que afecta al dinero y una interpretación errónea puede causar un problema. Para «abre la ayuda», al ser una acción de bajo riesgo, puede abrirla directamente. La decisión debe tener en cuenta tanto la confianza del modelo como el impacto de la acción.

c)
Habría que comprobar, al menos: 1 el ruido ambiental de distinta intensidad; 2 los distintos acentos y formas de pronunciar la orden; 3 diferentes micrófonos o dispositivos; y 4 las distintas distancias y niveles de volumen de la voz. También conviene probar con dispositivos de poca potencia y condiciones reales de uso. Que funcione en el portátil del desarrollador no garantiza que generalice a usuarios, equipos y entornos diferentes; el modelo debe validarse y probarse con condiciones representativas antes de desplegarse.

Ejercicio 2

a)
«Abrir Camara» -> “OPEN_CAMERA” -> porque tras formatearlo contiene “cámara” y “abrir”
«ABRIR la cámara» -> "UNKNOWN_INTENT" -> porque el programa no está preparado para el acento
«activa la ayuda por favor» -> "UNKNOWN_INTENT" -> porque no contiene “activar”, sino “activa”
«necesito ayuda» -> "UNKNOWN_INTENT" -> contiene “ayuda” pero no contiene “activar”
null -> lanzara un error porque intetara hacer lowercase a un null

b)

```
import java.text.Normalizer;

public class RobustCommandInterpreter implements CommandInterpreter {
    @Override
    public String detectIntent(String inputText) {
        if (inputText == null || inputText.isBlank()) {
            return "EMPTY_INPUT";
        }

        String normalized = Normalizer
                .normalize(inputText.toLowerCase().trim(), Normalizer.Form.NFD)
                .replaceAll("\\p{M}", "");

        if (normalized.contains("camara")
                && (normalized.contains("abrir")
                || normalized.contains("abre")
                || normalized.contains("enciende"))) {
            return "OPEN_CAMERA";
        }

        if (normalized.contains("ayuda")) {
            return "ENABLE_HELP_MODE";
        }

        if (normalized.contains("salir")) {
            return "CLOSE_APP";
        }

        return "UNKNOWN";
    }
}
```

c)

```
public class RobustCommandInterpreterTest {
    public static void main(String[] args) {
        CommandInterpreter interpreter = new RobustCommandInterpreter();

        String[] inputs = {
            "Abrir Camara",
            " ABRE la cámara ",
            "enciende la cámara frontal",
            "activa la ayuda por favor",
            "necesito salir ahora",
            "hola",
            null,
            "   "
```

```
        };

        String[] expected = {
            "OPEN_CAMERA",
            "OPEN_CAMERA",
            "OPEN_CAMERA",
            "ENABLE_HELP_MODE",
            "CLOSE_APP",
            "UNKNOWN",
            "EMPTY_INPUT",
            "EMPTY_INPUT"
        };

        System.out.printf("%-32s | %-20s | %-20s%n",
                "Entrada", "Esperada", "Obtenida");
        System.out.println("-".repeat(78));

        for (int i = 0; i < inputs.length; i++) {
            String result = interpreter.detectIntent(inputs[i]);
            System.out.printf("%-32s | %-20s | %-20s%n",
                    String.valueOf(inputs[i]), expected[i], result);
        }
    }
}
```

Entrada	Intención esperada	Intención obtenida	¿Coincide?
"Abrir Camara"	OPEN_CAMERA	OPEN_CAMERA	Sí
" ABRE la cámara "	OPEN_CAMERA	OPEN_CAMERA	Sí
"enciende la cámara frontal"	OPEN_CAMERA	OPEN_CAMERA	Sí
"activa la ayuda por favor"	ENABLE_HELP_MODE	ENABLE_HELP_MODE	Sí
"necesito salir ahora"	CLOSE_APP	CLOSE_APP	Sí
"hola"	UNKNOWN_INTENT	UNKNOWN_INTENT	Sí
null	EMPTY_INPUT	EMPTY_INPUT	Sí
" "	EMPTY_INPUT	EMPTY_INPUT	Sí

d)
Una frase razonable que puede clasificarse mal es «¿Podrías mostrarme la cámara, por favor?». No contiene «abrir», «abre» ni «enciende», por lo que la regla no la reconoce como OPEN_CAMERA aunque la intención del usuario sea abrir la cámara. Esto demuestra que las reglas basadas en palabras concretas pueden ser frágiles ante distintas formas de expresar la misma intención.

La pantalla debería depender de la interfaz CommandInterpreter y no de la implementación concreta. Así, la interfaz solo envía el texto y recibe una intención estable, sin conocer cómo se obtiene. Si se sustituye la clase por otra conectada a un modelo de procesamiento del lenguaje natural (PLN), se cambia la implementación, pero se mantiene el mismo contrato. Esto reduce el acoplamiento y permite probar y mantener cada parte por separado.

Ejercicio 3

a)

Número de ejemplos (n)	**Entrenamiento (60 %)**	**Validación (20 %)**	**Prueba (resto)**	**¿Algún conjunto vacío?**
2	1	0	1	si, validación
6	4	1	1	no
9	5	2	2	no
12	7	2	3	no
25	15	5	5	no

b)
Si se elimina Collections.shuffle, los ejemplos se repartirán respetando su orden original. Como están agrupados por intención, algunos conjuntos podrían contener sobre todo una o dos intenciones y no representar bien el conjunto de datos. Barajar antes de dividir ayuda a obtener subconjuntos más representativos. La semilla 42 hace que el barajado sea reproducible: al ejecutar el programa con los mismos datos se obtiene el mismo orden y se pueden comparar resultados.

c)
subList devuelve una vista de una parte de la lista original, no una lista independiente. Envolver cada subList en new ArrayList<>(...) crea una copia independiente, de modo que los cambios posteriores en la lista original no afecten a los tres conjuntos. Comprobar que ninguno está vacío evita un fallo silencioso: el programa podría continuar sin lanzar una excepción y parecer que ha validado o probado el modelo cuando en realidad no hay ejemplos con los que hacerlo.

d)
Le respondería que esa diferencia puede indicar sobreajuste. El modelo obtiene un 99 % con los datos de entrenamiento, pero solo un 61 % con datos de validación que no ha visto durante el ajuste. Por tanto, puede haberse adaptado demasiado a los ejemplos de entrenamiento y no generalizar bien a casos nuevos. No debería considerarse listo solo por el resultado de entrenamiento: hay que analizar los errores, validar con datos representativos y evaluar finalmente con un conjunto de prueba independiente.

